

Guia para IA

Continuidade do Projeto PyFlow RPA

Documento de orientacao para uma IA assistente continuar o desenvolvimento do PyFlow RPA a partir do ponto atual.

Maio 2026

Secao 1 — O que e este projeto?

PyFlow RPA e uma ferramenta desktop de automacao visual (estilo UiPath/n8n) feita em Python. O usuario arrasta blocos para um canvas com nos conectados por setas bezier, configura parametros e executa fluxos de automacao sem escrever codigo.

Componente	Tecnologia / Detalhe
Interface	PySide6 (Qt)
Canvas	QGraphicsView com nos draggaveis, portas Sucesso/Erro, conexoes bezier
Automacao web	Selenium + ChromeDriver
API local	FastAPI + Uvicorn (servidor embutido, porta padrao 8080)
Fluxos salvos	JSON na pasta flows/
Tema	Catppuccin Mocha (dark only — modo claro foi removido)
Python	3.11+
Caminho raiz	C:\Users\Gleudson\pasta4\Python\pyflow\pyflow\

Secao 2 — Os 5 arquivos que voce SEMPRE modifica ao adicionar um nov

FILE 1 — blocks//nome_do_bloco.py (CRIAR)

- Todo bloco herda de BaseBlock (blocks/base_block.py).
- Atributos de classe obrigatorios: name, description, category, params_schema
- Metodo execute(self, params: dict) -> dict
- Retorno padrao: {"success": bool, "message": str}
- Retorno com dados: {"success": True, "message": "...", "data": {...}}
- Categorias validas: "Navegador", "Controle", "Arquivos", "Integracao", "Sistema"
- Sempre inicie execute() chamando self.validate_params(params)

FILE 2 – ui/block_panel.py (MODIFICAR)

- Adicionar import da nova classe.
- Adicionar a classe na lista AVAILABLE_BLOCKS na posicao correta por categoria.

FILE 3 – ui/node_canvas.py (MODIFICAR)

- Adicionar import + "NomeDaClasse": NomeDaClasse no dicionario BLOCK_REGISTRY.
- Se for bloco de escopo: tambem atualizar _SCOPE_COLORS, _OPEN_TYPES, _CLOSE_TYPES.

FILE 4 – engine/runner.py (MODIFICAR – so para blocos de controle de fluxo)

- Se o bloco tiver logica de loop/condicao.
- _SKIP_TYPES = frozenset com marcadores de fim.
- NUNCA faca _run_sub iterar steps manualmente — deve sempre chamar self.run(steps).

FILE 5 – flows/teste_nome_do_bloco.json (CRIAR)

- Toda funcionalidade nova ganha um fluxo de teste JSON em flows/ com prefixo "teste_".

Secao 3 — Arquitetura do Canvas de Nos (MUITO IMPORTANTE)

O canvas foi COMPLETAMENTE REESCRITO para o modelo de nos estilo n8n. Arquivo principal: ui/node_canvas.py

Classe	Descricao
PortItem	Porta de conexao. port_type in ("input", "success", "error"). Cores: input=#8b8fa8 success=#a6e3a1 error=#f38ba8
ConnectionItem	Seta bezier entre portas. Colorida conforme porta de origem.
NodeItem	No visual. Tres portas: in_port (esq), success_port (dir-sup), error_port (dir-inf). Suporta pin de output e estado running/success/error.
NodeScene	Gerencia nos e conexoes. Metodo get_graph() retorna grafo para o runner.
_NodeView	QGraphicsView com RubberBandDrag (selecao multipla), pan por botao do meio, zoom por scroll.
NodeCanvas	Widget publico. API identica ao Canvas original para compatibilidade.

Interacao do usuario no canvas

- Arrastar no espaco vazio: selecao multipla (rubber band roxo)
- Arrastar com botao do meio do mouse: pan
- Scroll: zoom in/out
- Del: apaga nos/conexoes selecionados
- Ctrl+D: duplica nos selecionados
- Ctrl+Z / Ctrl+Y: desfaz / refaz
- Duplo clique no no: edita parametros
- Clique direito no no: menu contextual (editar, executar, fixar output, duplicar, remover)
- Snap to grid: nos se alinham ao grid de 24px automaticamente

Secao 4 — Execucao Condicional por Saida (Sucesso/Erro)

Porta	Cor	Ativada quando...
success_port	Verde (#a6e3a1)	execute() retorna {"success": True, ...}
error_port	Vermelha (#f38ba8)	execute() retorna {"success": False, ...}

Runner — run_graph() em engine/runner.py

- Detecta raizes (nos sem entradas)
- Executa cada no, le o resultado success/fail
- Segue a aresta correta (next_success ou next_error)
- Set visited previne loops infinitos

Serializacao JSON dos nos

```
_next_success  - ID do no de destino em caso de sucesso
_next_error    - ID do no de destino em caso de erro
_next (legado) - tratado como _next_success (retrocompatibilidade)
Arquivo de teste: flows/demo_condicional.json
```

Secao 5 — Arquitetura de Controle de Fluxo

Os blocos Loop, Para Cada e Se usam o modelo de marcadores de escopo. O Runner detecta marcadores automaticamente — nao sao usados blocks_count.

Bloco de abertura	Marcador de fim
Loop (Repetir)	EndLoopBlock
Para Cada (For Each)	EndForEachBlock
Condicao (Se)	ElseBlock (opcional) -> EndIfBlock

Todos os marcadores de fim estao em _SKIP_TYPES no runner.

Chaves especiais retornadas pelo bloco de abertura

```
{"loop": True, "times": 3}
{"foreach": True, "items": [...]}
{"if_result": True/False}
```

O runner detecta a chave, chama _find_scope_end() para achar o marcador e executa os blocos internos via _run_sub(). Aninhamento funciona porque _run_sub chama self.run() recursivamente.

Secao 6 — Onde ficam as variaveis do fluxo

Operacao	Codigo
Ler variavel	<code>ExtractTextBlock._context.get("minha_var", "")</code>
Escrever variavel	<code>ExtractTextBlock._context["minha_var"] = "valor"</code>
Contexto (arquivo)	<code>blocks/browser/extract_text.py</code> — dict estatico <code>_context</code>
Sintaxe nos params	<code>{{nome_da_variavel}}</code> — substituido por <code>resolve_params()</code> no runner
Assets/credenciais	<code>{{ASSET:chave}}</code> — buscado no <code>assets.json</code> via <code>AssetManager</code>

Secao 7 — Estado atual do projeto

Item	Valor
Total de blocos	49
Categoria Navegador	21 blocos
Categoria Controle	15 blocos (Loop, ForEach, Se, Else, FimLoop, FimForEach, FimSe...)
Categoria Arquivos	7 blocos
Categoria Integracao	3 blocos
Categoria Sistema	4 blocos
API	FastAPI + Uvicorn, porta 8080 (auto-detect se ocupada)
Tema	Catppuccin Mocha (dark only)
Canvas	Node-based com portas Sucesso/Erro, bezier, snap to grid
Selecao multipla	RubberBandDrag + mover grupo
Atalhos	Del, Ctrl+D, Ctrl+Z, Ctrl+Y
Icone do app	<code>assets/icon.png</code> + <code>assets/icon.ico</code> (gerado por <code>assets/generate_icon.py</code>)
Fluxos de exemplo	30+ em flows/

Secao 8 — Armadilhas — erros comuns para NUNCA cometer

ARMADILHA 1: Nao use blocks_count nos blocos de controle

Loop, Para Cada e Se nao usam mais blocks_count.
O Runner detecta os marcadores automaticamente.

ARMADILHA 2: _run_sub deve sempre chamar self.run()

Se _run_sub iterar os steps manualmente, aninhamentos vao quebrar.
Sempre delegar a recursao para self.run(steps).

ARMADILHA 3: Modo claro foi removido completamente

theme_manager.py tem apenas o tema escuro Catppuccin Mocha.
Nao existe toggle de tema.

ARMADILHA 4: A API usa FastAPI, nao Flask

api_server.py usa fastapi + uvicorn.
Flask foi removido do projeto.

ARMADILHA 5: Nao crie outro dict de contexto

Nao crie variavel global paralela para estado do fluxo.
Use sempre ExtractTextBlock._context.

ARMADILHA 6: Pan no canvas usa botao do MEIO

O botao esquerdo no espaco vaziao faz selecao multipla (rubber band).
Pan = botao do meio do mouse.

ARMADILHA 7: Canvas e node_canvas.py, nao canvas.py

O arquivo foi renomeado/substituido.
O import em main_window.py e: from ui.node_canvas import NodeCanvas as Canvas

Secao 9 — Checklist rapido — adicionando um bloco simples

#	Acao	Detalhe
1	Criar	blocks//nome_do_bloco.py herdando de BaseBlock
2	Preencher	name, description, category, params_schema na classe
3	Implementar	execute() retornando {"success": bool, "message": str}
4	Registrar	Importar e adicionar em ui/block_panel.py -> AVAILABLE_BLOCKS
5	Registrar	Importar e adicionar em ui/node_canvas.py -> BLOCK_REGISTRY
6	Testar	Criar flows/teste_nome_do_bloco.json

Secao 10 — Checklist — adicionando um bloco de escopo (Loop, Se, etc.)

#	Arquivo / Etapa	O que fazer
1	Bloco de abertura	Retorna data com chave identificadora especial
2	Bloco de fim	EndMeuBloco — execute() retorna so success/message
3	runner.py	Adicionar EndMeuBloco em _SKIP_TYPES
4	runner.py	Detectar data.get("meu_identificador") e usar _find_scope_end()
5	node_canvas.py	Adicionar MeuBloco em _SCOPE_COLORS e _OPEN_TYPES
6	node_canvas.py	Adicionar EndMeuBloco em _CLOSE_TYPES
7	block_panel.py	Adicionar ambos em AVAILABLE_BLOCKS
8	node_canvas.py	Adicionar ambos em BLOCK_REGISTRY
9	Teste	Criar flows/teste_meu_bloco.json

Secao 11 — Dicas de comunicacao com o usuario

- Confirmar o que sera feito antes de modificar arquivos.
- Pedir para ver o arquivo antes de editar.
- Toda funcionalidade nova deve ganhar fluxo de teste JSON em flows/.
- Se precisar de arquivo nao fornecido, dizer exatamente qual e por que.
- O dono do projeto chama-se Gleidson.
- O projeto e em portugues (nomes de blocos, mensagens, labels — tudo em pt-BR).
- O usuario prefere confirmacao antes de mudancas grandes.

Documento gerado para handoff de desenvolvimento — PyFlow RPA | Python 3.11+ | PySide6 | FastAPI | Maio 2026